Search Tree Performance Comparison
BST and Splay Tree

Group number: _____

October 17, 2026

Abstract

本实验实现并比较非平衡二叉搜索树（BST）和伸展树（Splay Tree）的插入、删除与查找。测试包括递增插入递增删除、递增插入逆序删除、随机插入随机删除三种序列。结果显示，BST 在有序数据下退化为链，而Splay Tree 通过伸展操作保持良好的摊还性能。

1 Introduction

BST 维护左子树键值小于根、右子树键值大于根的性质。普通BST 不主动调整树高，有序插入时可能退化为高度为N 的链。Splay Tree 在每次访问后将目标节点（或最后访问节点）伸展到根，单次最坏复杂度为$O(N)$，但任意序列的摊还复杂度为$O(\log N)$。本报告只讨论这两种结构，所有键值互不相同。

2 Data Structure and Algorithm Specification

2.1 BST

节点由整数键值和两个子指针组成。插入从根递归下降，空指针处创建节点；重复键忽略。查找沿比较结果下降。删除分为叶节点、单子节点和双子节点三种情况；双子节点使用右子树最小节点（中序后继）替换后递归删除。

```
TreeNode *DeleteNode(TreeNode *Root, int val) {
    if (Root == NULL) return NULL;
    if (val < Root->val) Root->Left = DeleteNode(Root->Left, val);
    else if (val > Root->val) Root->Right = DeleteNode(Root->Right, val);
    else if (Root->Left == NULL) { TreeNode *p=Root->Right; free(Root); return p; }
    else if (Root->Right == NULL) { TreeNode *p=Root->Left; free(Root); return p; }
    else { TreeNode *p=GetMin(Root->Right); Root->val=p->val;
           Root->Right=DeleteNode(Root->Right,p->val); }
    return Root;
}
```

2.2 Splay Tree

Splay 使用top-down 算法。临时头结点把访问路径拆为左、中、右三部分，循环执行zig-zig 和链接，最后重新拼接。插入先伸展目标键，再把新节点置于根；删除先伸展目标，若存在则将左子树最大节点伸展到根并接回原右子树。

```
SplayTree *Splay_Delete(SplayTree *Root, int val) {
    if (Root == NULL) return NULL; Root=Splay(Root,val);
    if (Root->val != val) return Root;
    SplayTree *old=Root, *right;
    if (Root->Left == NULL) Root=Root->Right;
    else { right=Root->Right; Root=Splay(Root->Left,val); Root->Right=right; }
```

```
    free(old); return Root;
}
```

3 Testing Results

3.1 Test design

`test.c` 先验证插入、查找、删除和删除后的集合内容，然后执行三种性能实验：Case 1 递增插入递增删除；Case 2 递增插入逆序删除；Case 3 随机插入随机删除。随机序列使用固定种子114514。当前代码实际接受$1000 \le N \le 20000$，提示中的100000 上限需要后续统一。

Table 1: 平均运行时间（秒）

N	Case 1		Case 2		Case 3	
	BST	Splay	BST	Splay	BST	Splay
1000	0.005	0.000	0.007	0.000	0.000	0.000
5000	0.105	0.000	0.209	0.001	0.001	0.001
10000	0.374	0.001	0.789	0.001	0.002	0.002
20000	1.641	0.001	3.303	0.000	0.005	0.005

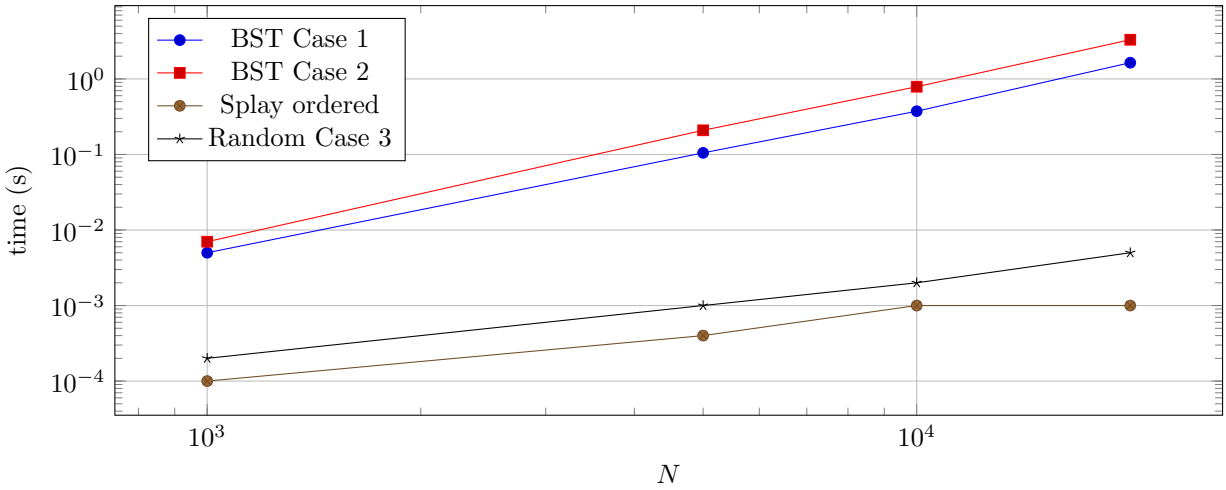


Figure 1: 运行时间趋势。零值计时点以小占位值绘图，表格保留原始输出。

4 Analysis and Comments

BST 的操作复杂度为$O(h)$。随机输入时树高通常为$O(\log N)$；有序插入时$h = O(N)$，因此Case 1 和Case 2 呈近似二次增长，逆序删除的遍历代价更大。Splay 单次操作最坏为$O(N)$，但摊还为$O(\log N)$，有序场景中访问局部性使其明显优于退化BST；随机场景下两者性能接近。Splay 不保存平衡因子，代价是树形随访问序列改变。`clock()` 的分辨率有限，表中0.000 不表示真实耗时为零。更严格的实验应提高精度、统一N 上限并重复测量取平均。

5 Conclusion

本实验完成了BST 和Splay Tree 的结构、插入、查找和删除。实验验证了普通BST 对有序输入敏感，而Splay Tree 对有序访问更稳定；随机输入下BST 实现更简单，Splay 则具有自适应优势。

6 Author list

BST implementation, Splay implementation, testing, and report: _____.

A Source Code Interface

实现位于项目根目录的`main.c`，实验驱动位于`test.c`；本报告未修改二者。